

Data Mining and Machine Learning

Assignment 3

Name: Abdullah Ayub | Roll No: BSDSF24M029

Polynomial Regression Using Gradient Descent

Weight-update equations and MSE comparison against the Normal Equation

1. Common setup

Every model is a polynomial in one variable fitted to the training pairs (x, R) with Gradient Descent (GD), implemented from scratch in NumPy. With m training examples and feature functions $\phi_j(x)$ ($\phi_0=1, \phi_j=x^j$), the hypothesis is $h_\theta(x)=\sum_j \theta_j \phi_j(x)$. The cost function is:

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})^2 = \frac{1}{2m} \|X\theta - y\|^2$$

Differentiating with respect to each weight gives:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)}) \phi_j(x^{(i)})$$

so the weight-update rule applied simultaneously to all j at every iteration is:

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)}) \phi_j(x^{(i)})$$

In vectorized form (X is the $m \times (d+1)$ feature matrix, no loop over examples):

$$\theta \leftarrow \theta - \frac{\alpha}{m} X^T (X\theta - y)$$

For degrees 2–6 the code standardizes the input, $z = (x - \mu)/\sigma$, using the training mean and standard deviation, and builds features $[1, z, z^2, \dots, z^d]$. The same μ and σ are reused on the test set. The linear model uses the raw x .

2. Weight-update equations per model

Each model below shows its hypothesis and one update equation per weight. All updates are applied simultaneously.

2.1 Linear model ($d = 1$, 2 weights)

$$h_\theta = \theta_0 + \theta_1 x$$

$$\theta_0 \leftarrow \theta_0 - \frac{\alpha}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})$$

$$\theta_1 \leftarrow \theta_1 - \frac{\alpha}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)}) (x^{(i)})$$

Hyperparameters: $\alpha = 0.01$, iterations = 1,000, input feature = x (raw), θ initialised to zeros.

2.2 Quadratic model (d = 2, 3 weights)

$$h_{\theta} = \theta_0 + \theta_1 z + \theta_2 z^2$$

$$\theta_0 \leftarrow \theta_0 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 \leftarrow \theta_1 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})$$

$$\theta_2 \leftarrow \theta_2 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^2$$

Hyperparameters: $\alpha = 0.01$, iterations = 5,000, input feature = z, θ initialised to zeros.

2.3 Cubic model (d = 3, 4 weights)

$$h_{\theta} = \theta_0 + \theta_1 z + \theta_2 z^2 + \theta_3 z^3$$

$$\theta_0 \leftarrow \theta_0 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 \leftarrow \theta_1 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})$$

$$\theta_2 \leftarrow \theta_2 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^2$$

$$\theta_3 \leftarrow \theta_3 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^3$$

Hyperparameters: $\alpha = 0.005$, iterations = 10,000, input feature = z, θ initialised to zeros.

2.4 4th-degree model (d = 4, 5 weights)

$$h_{\theta} = \theta_0 + \theta_1 z + \theta_2 z^2 + \theta_3 z^3 + \theta_4 z^4$$

$$\theta_0 \leftarrow \theta_0 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 \leftarrow \theta_1 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})$$

$$\theta_2 \leftarrow \theta_2 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^2$$

$$\theta_3 \leftarrow \theta_3 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^3$$

$$\theta_4 \leftarrow \theta_4 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^4$$

Hyperparameters: $\alpha = 0.002$, iterations = 15,000, input feature = z, θ initialised to zeros.

2.5 5th-degree model (d = 5, 6 weights)

$$h_{\theta} = \theta_0 + \theta_1 z + \theta_2 z^2 + \theta_3 z^3 + \theta_4 z^4 + \theta_5 z^5$$

$$\theta_0 \leftarrow \theta_0 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 \leftarrow \theta_1 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})$$

$$\theta_2 \leftarrow \theta_2 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^2$$

$$\theta_3 \leftarrow \theta_3 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^3$$

$$\theta_4 \leftarrow \theta_4 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^4$$

$$\theta_5 \leftarrow \theta_5 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^5$$

Hyperparameters: $\alpha = 0.001$, iterations = 20,000, input feature = z, θ initialised to zeros.

2.6 6th-degree model (d = 6, 7 weights)

$$h_{\theta} = \theta_0 + \theta_1 z + \theta_2 z^2 + \theta_3 z^3 + \theta_4 z^4 + \theta_5 z^5 + \theta_6 z^6$$

$$\theta_0 \leftarrow \theta_0 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 \leftarrow \theta_1 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})$$

$$\theta_2 \leftarrow \theta_2 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^2$$

$$\theta_3 \leftarrow \theta_3 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^3$$

$$\theta_4 \leftarrow \theta_4 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^4$$

$$\theta_5 \leftarrow \theta_5 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^5$$

$$\theta_6 \leftarrow \theta_6 - \frac{\alpha}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) (z^{(i)})^6$$

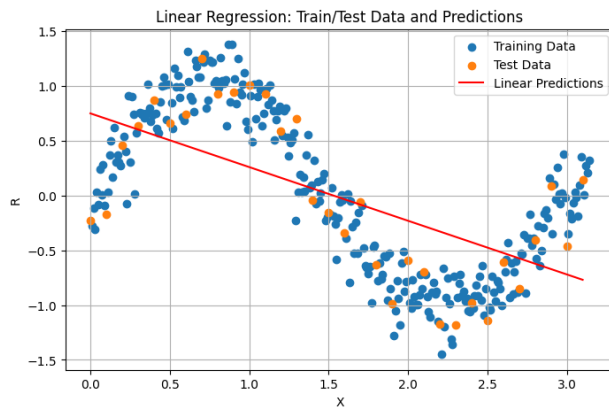
Hyperparameters: $\alpha = 0.0005$, iterations = 25,000, input feature = z, θ initialised to zeros.

3. Gradient Descent results

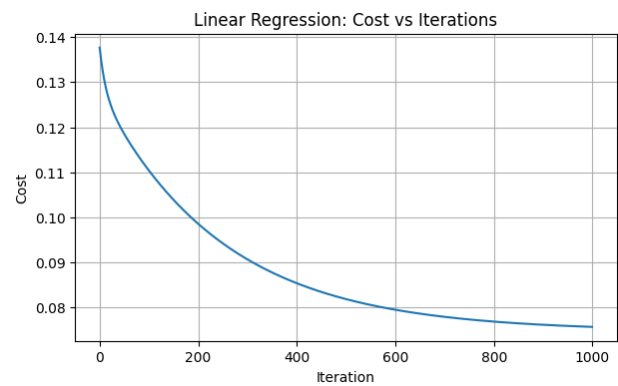
Test-set error from the notebook. Note that the notebook's "MSE" is computed with the cost-function convention, $\text{np.mean}((\text{pred} - y)^2)/2$, i.e. it equals $J(\theta) = \frac{1}{2} \cdot (\text{mean squared error})$. The plain MSE is therefore twice the value shown.

Model	alpha	Iterations	Test MSE ($J = \text{MSE}/2$)	Plain MSE ($2 \times J$)
Linear	0.01	1,000	0.157080	0.314160
Quadratic	0.01	5,000	0.163021	0.326042
Cubic	0.005	10,000	0.025780	0.051560
4th-degree	0.002	15,000	0.024449	0.048898
5th-degree	0.001	20,000	0.038858	0.077716
6th-degree	0.0005	25,000	0.046128	0.092256

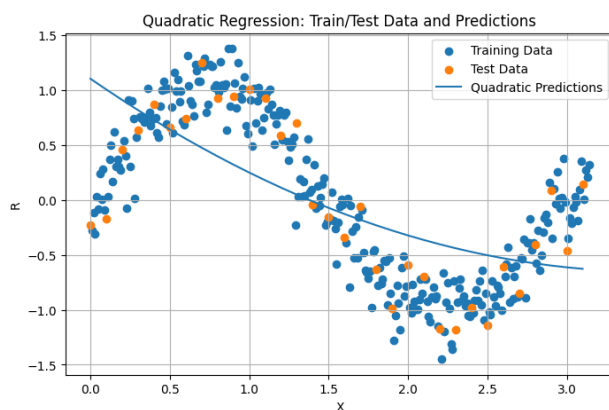
Fitted curves (left) and cost vs. iterations (right) for each model:



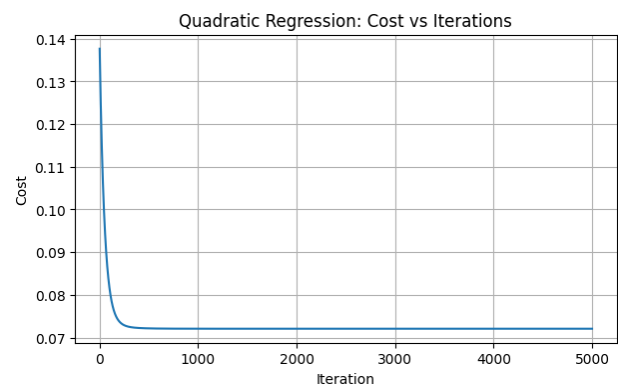
Linear: train/test data and predictions



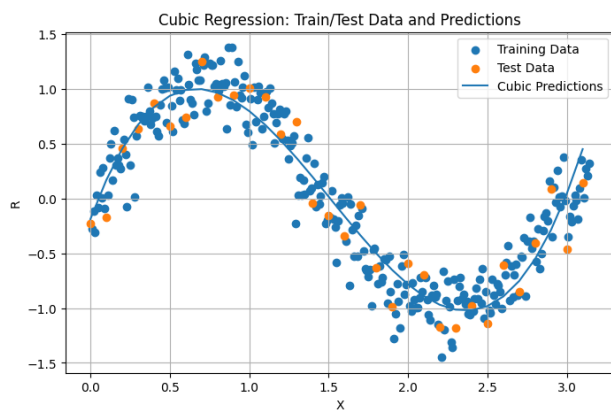
Linear: cost vs iterations



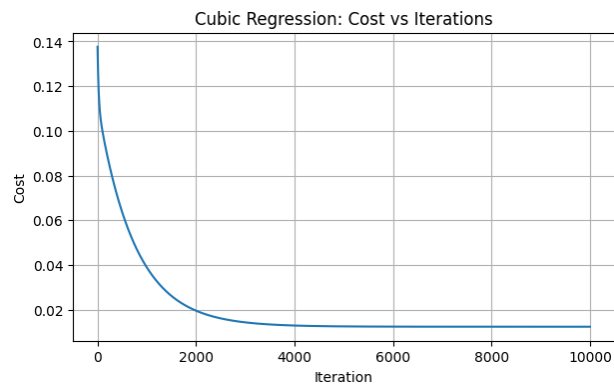
Quadratic: train/test data and predictions



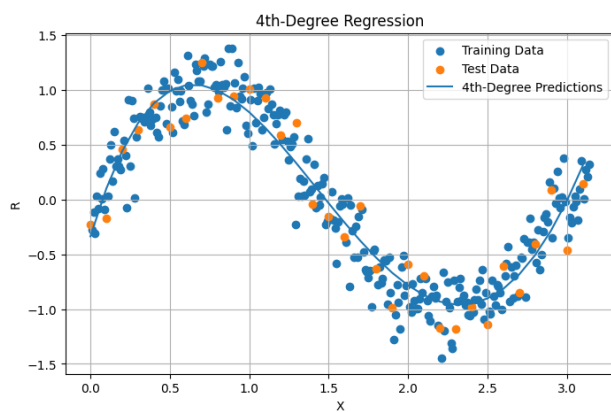
Quadratic: cost vs iterations



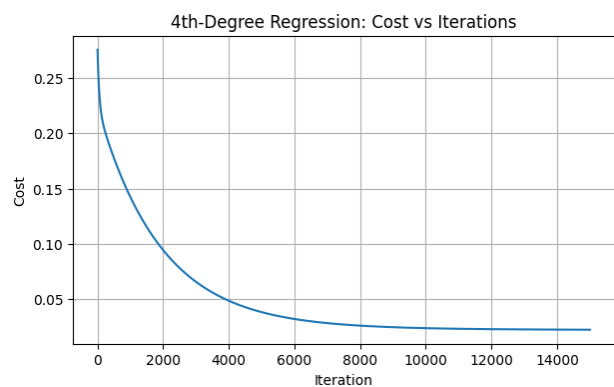
Cubic: train/test data and predictions



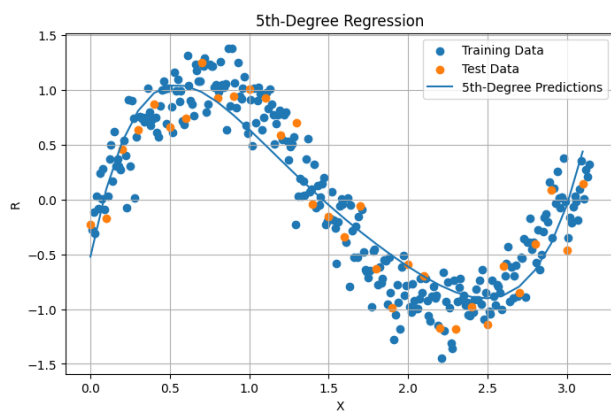
Cubic: cost vs iterations



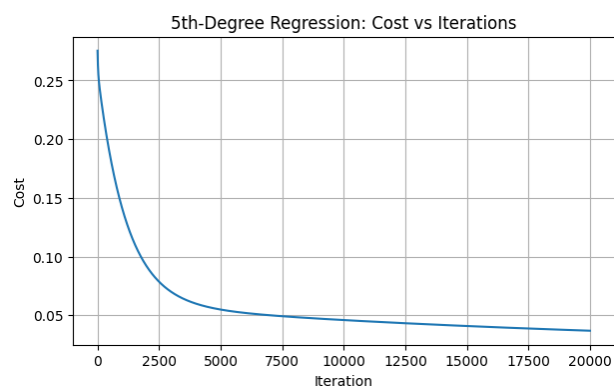
4th-degree: train/test data and predictions



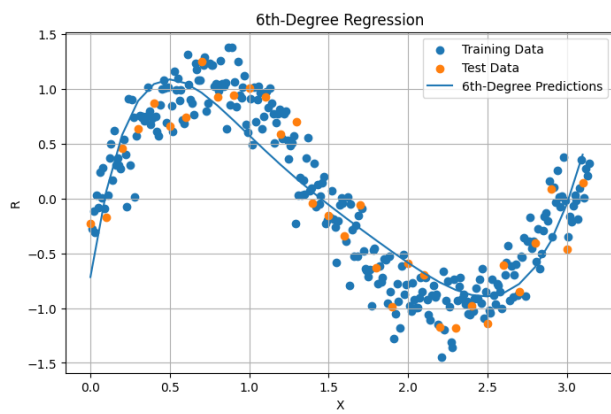
4th-degree: cost vs iterations



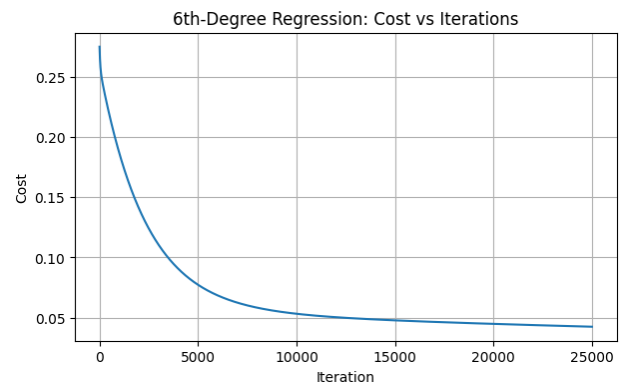
5th-degree: train/test data and predictions



5th-degree: cost vs iterations



6th-degree: train/test data and predictions



6th-degree: cost vs iterations

4. Gradient Descent vs. Simultaneous Equations

In the previous assignment the weights were found by solving the simultaneous (normal) equations in closed form:

$$X^T X \theta = X^T y \Rightarrow \theta = (X^T X)^{-1} X^T y$$

This gives the exact minimiser of the training error in one step, whereas GD approaches it iteratively. If GD has fully converged, both approaches give the same weights and the same test MSE; any gap means GD stopped early. The previous assignment reports plain MSE = mean((prediction – R)²), so for a like-for-like comparison the GD values below are the notebook's printed values multiplied by 2 (the GD notebook prints J = MSE/2). All values are test-set MSE on the same train/test CSV files.

Model	GD test MSE	Simultaneous-Eq test MSE (previous)	Difference (GD - SE)
Linear	0.314160	0.315932	-0.001772
Quadratic	0.326042	0.326042	+0.000000
Cubic	0.051560	0.051542	+0.000018
4th-degree	0.048898	0.050025	-0.001127
5th-degree	0.077716	0.044207	+0.033509
6th-degree	0.092256	0.044509	+0.047747

The simultaneous-equation values are those printed by the previous assignment's notebook; re-solving on the standardized GD features reproduces them to six decimals.

5. Discussion

Agreement for low degrees. For the linear, quadratic and cubic models, GD and the simultaneous equations give almost identical test MSE (0.3142 vs 0.3159, 0.3260 vs 0.3260, 0.05156 vs 0.05154), so GD converged to essentially the same solution. The small differences for the linear model and the 4th-degree model (0.0489 vs 0.0500) are GD stopping slightly short of the exact minimiser; GD scoring a bit lower on the 32 test points is chance, not a better fit.

Degrees 5 and 6: GD had not converged. GD gives 0.0777 and 0.0923, but the exact solutions give 0.0442 and 0.0445. So the rise in GD error at higher degrees is not overfitting: the exact higher-degree fits generalise as well as or better than the cubic (0.0515). The cause is conditioning: the polynomial features are strongly correlated (cond($X^T X$) grows from about 1 for the linear model to about 9,600 for degree 6, on standardized features), so GD needs a small α and far more iterations along the flat directions of the cost surface. 20,000–25,000 iterations were still not enough.

Model complexity. The linear and quadratic models underfit (test MSE about 0.32). Error drops sharply at the cubic model (0.052) and improves only marginally beyond it with exact solutions (0.0500, 0.0442, 0.0445), so a cubic is sufficient and degrees 5–6 add little. The quadratic is no better than the linear model, so the data needs an odd (cubic) term.

Trade-offs. The simultaneous equations are exact, need no α or iteration count, and are instant for these small problems, but they require inverting $X^T X$, which becomes ill-conditioned for high degrees and costs $O(n^3)$ in the number of features. GD scales to large feature counts but needs standardisation, a tuned α and enough iterations.

Conclusion. The cubic model is the best simple choice, and any of degrees 3–6 are within a similar error range when solved exactly. To make GD match the simultaneous equations at degrees 5–6, increase the iterations substantially or raise α carefully.